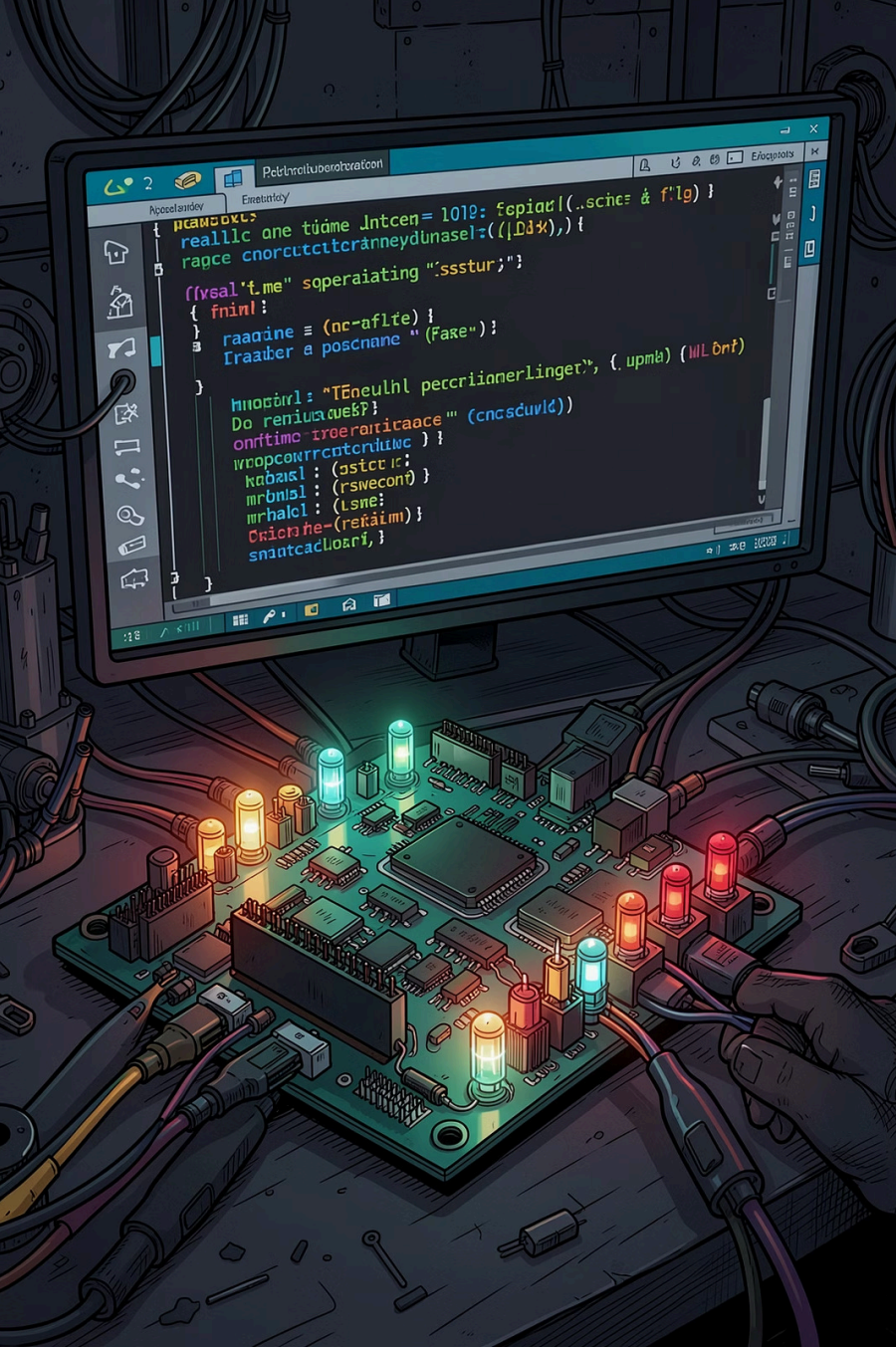




FreeRTOS pada Embedded Systems

Panduan teknis komprehensif tentang multitasking real-time pada microcontroller — mencakup manajemen task, mekanisme scheduling, keamanan memori, komunikasi antar-task, serta pertimbangan spesifik platform untuk STM32 dan ESP32. Dirancang untuk engineer embedded dan mahasiswa dengan dasar pemrograman C dan pengetahuan microcontroller.

FREERTOS

STM32

ESP32

REAL-TIME SYSTEMS

RTOS Fundamentals & Definisi Task

Apa Itu RTOS?

Real-Time Operating System menjamin **waktu respons yang dapat diprediksi dan terbatas** — sifat utama yang membedakannya dari OS tujuan umum. FreeRTOS bersifat open-source, hanya membutuhkan sekitar **10KB RAM**, dan dilengkapi **preemptive scheduler** yang memastikan task berprioritas tinggi selalu dieksekusi sebelum task berprioritas lebih rendah.

Task dengan prioritas lebih tinggi dapat **menginterupsi task dengan prioritas lebih rendah kapan saja**, sehingga RTOS ideal untuk sistem embedded yang harus merespons berbagai peristiwa penting secara bersamaan dan deterministik — sampling sensor, protokol komunikasi, alarm keselamatan.

Apa Itu Task?

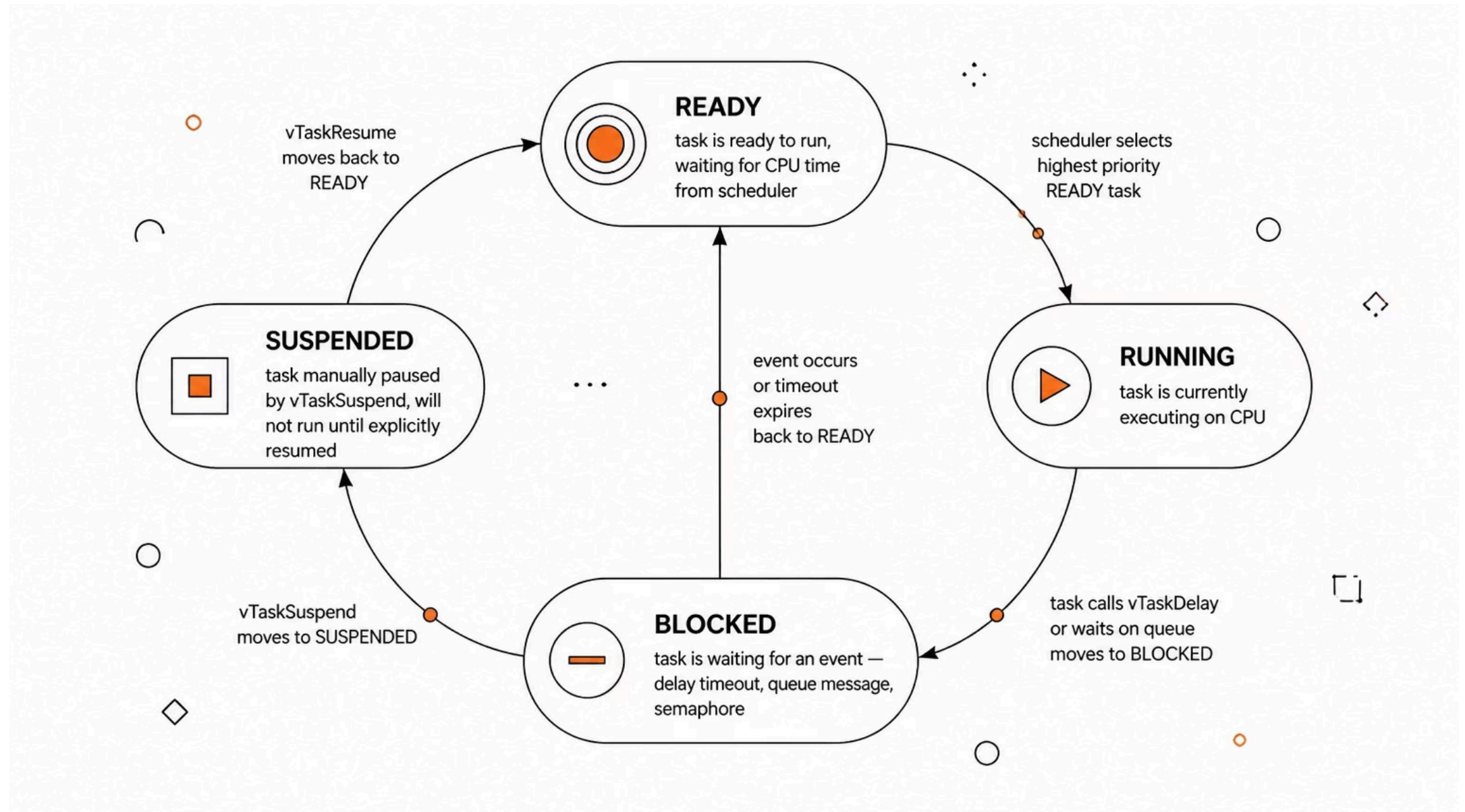
Task adalah unit eksekusi independen dengan **stack** sendiri (variabel lokal, return address, register yang tersimpan), **TCB** (Task Control Block — metadata yang mencakup state, priority, dan stack pointer), serta **entry function** yang harus berjalan dalam infinite loop.

Setiap task dieksekusi seolah-olah memiliki **akses CPU khusus**. Illusi paralelisme tercipta karena scheduler dengan cepat berpindah antar-task. Aturan penting:

- **Jangan pernah return** dari task entry function — function tersebut harus loop selamanya
- Selalu panggil `vTaskDelay()` untuk memberi waktu CPU ke task lain
- Lakukan inisialisasi satu kali sebelum masuk ke infinite loop
- Gunakan blocking API calls untuk menghindari busy-waiting

Status & Siklus Hidup Task

Setiap task FreeRTOS berada dalam salah satu dari empat status. Scheduler mengelola transisinya secara otomatis — memahami siklus hidup ini sangat penting untuk men-debug masalah timing, masalah prioritas, dan system hang.



Deskripsi Status

- **READY** — Task layak dijalankan; menunggu scheduler memberikan waktu CPU
- **RUNNING** — Task sedang mengeksekusi instruksi di CPU
- **BLOCKED** — Task menunggu event eksternal: timeout delay, pesan queue, atau semaphore
- **SUSPENDED** — Task dijeda secara manual melalui `vTaskSuspend()`; tidak akan berjalan sampai secara eksplisit di-resume

Alur Siklus Hidup Umum

`vTaskCreate()` → **READY** → Scheduler memilih task **READY** dengan prioritas tertinggi → **RUNNING** → Task dipreempt atau memanggil delay → **BLOCKED** → Event terjadi atau timeout berakhir → **READY** → siklus berulang.

Task juga dapat dipindahkan ke **SUSPENDED** kapan saja melalui `vTaskSuspend()` dan dikembalikan ke **READY** melalui `vTaskResume()`. Dari ISR, gunakan `xTaskResumeFromISR()`.

Prioritas, Preemption & Penjadwalan

Tingkat Prioritas

FreeRTOS mendukung tingkat prioritas dari **0 (idle) hingga nilai maksimum yang dikonfigurasi**. Semakin tinggi angkanya, semakin urgen scheduler memperlakukan task tersebut. Ketika task dengan prioritas lebih tinggi menjadi READY, task itu **segera melakukan preempt** terhadap task RUNNING dengan prioritas lebih rendah — context switch terjadi dalam hitungan mikrodetik.

Task dengan **prioritas yang sama** berbagi waktu CPU melalui **time-slicing**: scheduler bergantian di antara task-task tersebut setiap tick, sehingga masing-masing mendapat porsi pemrosesan yang setara.

Rate Monotonic Scheduling

Teori penjadwalan real-time formal yang diterapkan di FreeRTOS: task dengan **periode lebih pendek mendapat prioritas lebih tinggi**, dan task dengan periode lebih panjang mendapat prioritas lebih rendah. Secara matematis, ini menjamin semua deadline terpenuhi, selama total utilisasi CPU tetap di bawah batas terhitung (~69% untuk banyak task).

Preemption dalam Aksi

Skenario: Task A (prioritas 3) sedang RUNNING. Task B (prioritas 4) menjadi READY — mungkin karena di-unblock oleh interrupt hardware. Scheduler **segera menginterupsi Task A**, menyimpan context register-nya, dan memindahkan eksekusi ke Task B. Task A baru melanjutkan ketika Task B blocking, suspend, atau selesai.

Inilah keunggulan inti dari **preemptive RTOS** dibanding desain cooperative atau bare-metal: task kritis selalu mendapat CPU terlebih dahulu, terlepas dari apa yang sedang dilakukan code berprioritas lebih rendah.

 **Wawasan Utama:** Preemption tidak terlihat oleh task itu sendiri — Task A tidak menyadari bahwa ia diinterupsi. Seluruh state register disimpan dan dipulihkan secara atomik oleh scheduler.

Keamanan Memori Stack & Manajemen Heap

Stack: Sumber Daya Paling Kritis

Setiap task memiliki **stack pribadi** yang menyimpan variabel lokal, parameter fungsi, alamat kembali, dan register CPU yang tersimpan. Overflow stack adalah penyebab paling umum dari **korupsi diam-diam dan crash yang sulit didiagnosis** pada sistem FreeRTOS.

Menghitung ukuran stack:

- Jumlahkan semua variabel lokal di seluruh pemanggilan fungsi bertingkat
- Tambahkan ruang untuk register yang disimpan saat context switch
- Tambahkan **margin keamanan 30%** minimal
- STM32: stack diukur dalam **word** (masing-masing 4 byte)
- ESP32: stack diukur dalam **byte** — perbedaan migrasi yang sangat penting!

Gunakan `uxTaskGetStackHighWaterMark()` untuk memantau **minimum stack bebas** yang tersisa. High Water Mark 234 dari 512 word berarti task telah menggunakan 278 word — aman, tetapi tetap pantau trennya dari waktu ke waktu.

Manajemen Heap

FreeRTOS menggunakan **heap dinamis** untuk mengalokasikan memori bagi task, queue, semaphore, dan timer saat runtime. Default `heap_4.c` menerapkan **first-fit allocator** dengan penggabungan blok bebas yang berdampingan.

Ukuran heap pada platform:

- **STM32:** Biasanya total RAM 10–20 KB — heap sangat terbatas
- **ESP32:** Hingga 520 KB SRAM — jauh lebih longgar

Praktik terbaik:

- Alokasikan semua task, queue, dan semaphore **sebelum scheduler dimulai**
- Gunakan **static allocation** untuk task yang kritis terhadap keselamatan agar menghilangkan risiko kegagalan alokasi saat runtime
- Selalu pasang `vTaskDelete()` dengan `free()` manual untuk setiap memori heap yang dialokasikan task secara internal
- Pantau heap bebas sebelum dan sesudah siklus create/delete untuk mendeteksi kebocoran memori



Deteksi Overflow Stack: Aktifkan

`configCHECK_FOR_STACK_OVERFLOW=2` untuk mendeteksi penimpaan sebelum merusak memori di sekitarnya. Implementasikan `vApplicationStackOverflowHook()` untuk mencatat fault, menetapkan error flag, atau memicu restart yang aman.

Tick, Mekanisme Delay & Context Switching

System Tick

FreeRTOS digerakkan oleh **interrupt system tick** periodik. Konfigurasi default adalah **1000 Hz (1 ms per tick)** pada STM32 dan **100 Hz (10 ms per tick)** pada ESP32. Semua fungsi timing dinyatakan dalam hitungan tick. Gunakan `pdMS_TO_TICKS(milliseconds)` untuk konversi yang portabel.

Dua Fungsi Delay

API Kontrol Tugas Lanjutan

FreeRTOS menyediakan seperangkat API yang kaya untuk manajemen tugas dinamis saat runtime. API ini memungkinkan sistem adaptif yang merespons kondisi yang berubah, menangani keadaan darurat, dan mengelola sumber daya secara efisien.



vTaskSuspend() / vTaskResume()

Menjeda dan melanjutkan secara manual tugas apa pun. Gunakan `NULL` untuk self-suspend. Dari ISR, gunakan `xTaskResumeFromISR()`. Tidak seperti delay, tugas yang suspended tidur **tanpa batas waktu** sampai secara eksplisit dilanjutkan. Kasus penggunaan: mode pemeliharaan, penghematan daya, gating eksekusi bersyarat.



vTaskDelete()

Menghapus tugas secara permanen dan **membebaskan memori stack**-nya. Gunakan `NULL` untuk self-delete. Sebelum menghapus, selalu lakukan pembersihan: tutup file handle, bebaskan memori heap apa pun yang dialokasikan tugas. Pantau heap sebelum/sesudah siklus create-delete — kebocoran akan terlihat sebagai free heap yang terus menurun.



vTaskPrioritySet()

Mengubah prioritas tugas secara dinamis saat runtime. Pola: tetapkan **base priority** untuk operasi normal, lalu sementara **menaikkan prioritas** saat keadaan darurat (misalnya respons fault heater pada prioritas 4), kemudian kembalikan ke base. Memungkinkan perilaku real-time yang adaptif tanpa membuat ulang tugas.



vTaskList() & vTaskGetRunTimeStats()

`vTaskList()` menghasilkan tabel terformat dari semua tugas: nama, state, prioritas, dan stack High Water Mark. `vTaskGetRunTimeStats()` melaporkan persentase penggunaan CPU per tugas. Gabungkan keduanya untuk **dasbor sistem yang komprehensif** — pantau dalam jendela 10 detik untuk melacak idle% CPU dan tren pertumbuhan stack.

Idle Task & Manajemen Daya

Idle Task (prioritas 0) berjalan ketika tidak ada tugas lain yang READY. Daftarkan callback kustom melalui `vApplicationIdleHook()`. Pada ARM Cortex-M, panggil `__WFI()` (Wait For Interrupt) di dalam hook untuk menempatkan CPU ke **mode sleep dalam**, sehingga konsumsi daya berkurang drastis. **Jangan pernah** menambahkan panggilan blocking di idle hook — itu akan menyebabkan deadlock pada seluruh sistem.

Perlindungan Watchdog Timer

hardware watchdog akan me-reset MCU jika sebuah tugas gagal "feed" dalam jendela timeout (biasanya 4 detik). Jika sebuah tugas hang, tugas tersebut berhenti melakukan feed → timeout → reset otomatis → sistem pulih. STM32 menggunakan **IWDG** melalui HAL; ESP32 memiliki **Task Watchdog Timer** bawaan. Seimbangkan timeout dengan hati-hati: terlalu singkat menyebabkan false trigger; terlalu lama menunda pemulihan. **Jangan pernah menonaktifkan watchdog di firmware produksi.**

ESP32 vs STM32: Perbandingan Platform

Kedua platform menjalankan FreeRTOS, tetapi berbeda secara signifikan dalam arsitektur, model memori, dan kapabilitas. Memahami perbedaan ini sangat penting saat mem-porting kode atau memilih perangkat keras.

Fitur	STM32 (mis. F103)	ESP32
Core	Single-core ARM Cortex-M, 72 MHz	Dual-core Xtensa LX6, 240 MHz masing-masing
RAM	~20 KB SRAM — sangat terbatas	520 KB SRAM — sangat lega
Tick Rate	1000 Hz (tick 1 ms)	100 Hz (tick 10 ms)
Stack Unit	Word (4 byte per word)	Byte — perbedaan migrasi yang kritis!
Core Affinity	Tidak berlaku (single-core)	xTaskCreatePinnedToCore() — pin ke Core 0, Core 1, atau floating
Watchdog	IWDG via HAL	Task Watchdog Timer (TWDT)
Typical Heap	10–20 KB	>500 KB

Strategi Core Affinity ESP32

Gunakan `xTaskCreatePinnedToCore(task, "name", stack, params, priority, &handle, core_id)` untuk mem-pin task:

- **Core 0:** penanganan I/O, Wi-Fi, Bluetooth stack
- **Core 1:** task yang intensif komputasi, pemrosesan sinyal
- **Floating:** task fleksibel yang dapat berjalan di salah satu core

Mem-pin task mengurangi overhead context switch dan mencegah cache thrashing. Gunakan `xPortGetCoreID()` untuk memverifikasi core tempat sebuah task berjalan. STM32 tidak memiliki padanan — semua task berbagi core tunggal.

Pertimbangan Migrasi

Saat mem-porting dari STM32 ke ESP32 (atau sebaliknya):

- Kalikan ukuran stack **4×** dan ubah dari word ke byte
- Sesuaikan delay berbasis tick: 1000 Hz → 100 Hz berarti nilai `pdMS_TO_TICKS()` berubah
- Heap ESP32 yang lebih besar memungkinkan alokasi dinamis lebih banyak; STM32 memerlukan alokasi statis yang agresif
- ESP32 dual-core memungkinkan eksekusi paralel sejati; STM32 sepenuhnya bergantung pada time-slicing

Masalah Kritis: Priority Inversion & Race Conditions

Priority Inversion

Priority inversion terjadi ketika tugas berprioritas tinggi secara tidak langsung terblokir oleh tugas berprioritas rendah — melanggar jaminan real-time. Skenario klasik:

→ Tugas berprioritas rendah mengambil mutex

Low memegang kunci resource bersama

→ Tugas berprioritas tinggi meminta mutex

High terblokir, menunggu Low melepaskan

→ Tugas berprioritas menengah menjadi READY

Medium mem-preempt Low (tidak ada konflik lock)

→ Medium berjalan tanpa henti

High masih terblokir — Medium secara efektif menunda High!

Solusi — Priority Inheritance: Saat High menunggu mutex yang dipegang Low, FreeRTOS **sementara menaikkan prioritas Low agar sama dengan High**. Low langsung berjalan, segera melepaskan mutex, lalu kembali ke prioritas aslinya. Aktifkan dengan `configUSE_MUTEXES=1`. Selalu gunakan mutex (bukan binary semaphore) untuk melindungi resource bersama.

Race Conditions

Race condition terjadi ketika dua tugas mengakses variabel bersama tanpa sinkronisasi, dan setidaknya salah satunya melakukan penulisan. Contoh: Producer menulis nilai 32-bit sementara Consumer membacanya secara bersamaan. Pada sebagian besar MCU, baca/tulis 32-bit **tidak atomic** — memerlukan beberapa instruksi load/store. Context switch di tengah operasi menyebabkan **torn read**: Consumer melihat nilai korup yang setengah diperbarui.

Gejala: Pada beban CPU rendah, korupsi bisa jarang terjadi atau bahkan tidak muncul. Pada beban tinggi dengan context switch yang sering, tingkat korupsi 5–10% mungkin terjadi. Ketidakcocokan checksum pada struktur data bersama adalah tanda yang jelas.

⚠ **Solusi (Module 09):** Lindungi semua variabel bersama dengan **Queue** (untuk pengiriman data) atau **Mutex** (untuk mutual exclusion). Jangan pernah membagikan variabel global mentah antar tugas tanpa primitive sinkronisasi.

Praktikum Langsung: 10 Latihan

Latihan bertahap dari pembuatan task dasar hingga debugging lanjutan — setiap latihan membangun latihan sebelumnya dan menghasilkan hasil yang dapat diamati serta diukur.

Lab 01 — Create Tasks

2 LED blink tasks (500ms, 1000ms, priority 2) + monitor task (priority 1). Amati LED toggling, output task list, dan stack HWM melalui serial.

Lab 02 — Priority

3 tasks High/Med/Low with loop counters. Pantau distribusi CPU. Kirim 'S' via serial untuk menukar prioritas — amati rearrangement preemption secara langsung.

Lab 03 — Timing

Task A: `vTaskDelay(100ms)` vs Task B: `vTaskDelayUntil(100ms)`. Kumpulkan 100 sampel. Pastikan DelayUntil mencapai presisi $\pm 1\text{ms}$ tanpa drift.

Lab 04 — Suspend

LED toggle 500ms. Button interrupt memanggil `vTaskSuspend()`. Tombol kedua memanggil `vTaskResume()`. Amati LED berhenti/berjalan — demo maintenance mode.

Lab 05 — Delete

Buat task, loop 20x, lalu `vTaskDelete()`. Pantau heap sebelum/sesudah — harapkan pemulihan ± 50 byte. Penurunan heap bertahap mengindikasikan leak.

Lab 06 — Stack Monitor

3 tasks dengan Small/Medium/Large stacks. Uji kedalaman rekursif 10/20/30. Pantau HWM%. Picu overflow pada Small di depth 50+. Implementasikan overflow hook.

Lab 07 — Affinity & Inversion

ESP32: Pin tasks Core 0/1/floating, cetak `xPortGetCoreID()`, ukur perbedaan waktu eksekusi. **STM32:** Demonstrasikan priority inversion, verifikasi bahwa mutex inheritance menyelesaikannya.

Lab 08 — CPU%

Aktifkan `configUSE_IDLE_HOOK=1`. Hitung panggilan idle hook. $\text{CPU\%} = 100 \times (\text{Total} - \text{Idle}) / \text{Total}$. Terapkan beban 0–95%, verifikasi korelasi linear. Konfirmasi penghematan daya `__WFI()`.

Lab 09 — Watchdog

Good task feed WDT secara normal; Bad task tidak pernah feed. Pantau timeout 30 detik, amati auto-reset recovery. STM32: IWDG via HAL. ESP32: Task WDT. Tangkap boot recovery log.

Lab 10 — Race Condition

Producer writes, Consumer reads shared variable **without mutex**. Deteksi corruption melalui checksum mismatch. Beban rendah: error mendekati nol. Beban tinggi: korupsi 5–10%. Solusi: Module 09 Queue/Mutex.